UNIVERSIDAD PRIVADA DOMINGO SAVIO
Facultad de Ingeniería
Carrera de Ingeniería de Sistemas

Desarrollo de Sistemas II

Actividad Académica N° 1: Modelado C4 y Base de Datos

Sistema de Pedidos Mayoristas para Laboratorio Químico Delta

Estudiante: Gustavo Rodríguez Vélez
Docente: Vladimir Wilmar Rojas Condori

Cochabamba, 8 de noviembre de 2026

Índice

Descripción del negocio y problema
Justificación y objetivos
Usuarios y requerimientos
Modelado UML
Modelado C4
Base de datos
Prototipo de interfaces
Tecnologías seleccionadas
Planificación del desarrollo
Repositorio GitHub
Conclusiones
Anexos

Descripción del negocio y problema

Descripción del negocio

Laboratorio Químico Delta es una empresa de Cochabamba, Bolivia, que fabrica medicamentos y los vende al por mayor a las farmacias de la ciudad. No atiende al público final: sus clientes son las farmacias, que le hacen pedidos con varios medicamentos a la vez y los reciben en su local.

Queremos dejar claro que se trata de un escenario ficticio, inventado para este proyecto académico. No representa a ninguna empresa real, aunque existan otras con un nombre parecido, y todos los datos que usamos (farmacias, medicamentos, precios, nombres y números) son datos de prueba.

De todo lo que hace un laboratorio elegimos un solo proceso para el proyecto: la recepción y el seguimiento de los pedidos mayoristas que hacen las farmacias. Es un proceso con un principio y un final claros, lo hacen personas concretas y deja información que se puede guardar, consultar y reportar.

Alcance del proyecto

El sistema se limita al ciclo de vida de un pedido, desde que una farmacia lo solicita hasta que lo recibe. Todo lo que no ayude a registrar o seguir un pedido queda fuera, para que el trabajo se pueda terminar y demostrar dentro del semestre.

Incluye:

- Farmacias, que son los clientes del laboratorio.
- El catálogo de medicamentos, organizado por categorías.
- Pedidos con su detalle de medicamentos y cantidades, y con un estado: pendiente, despachado, entregado o anulado.
- Usuarios del sistema con roles.
- El descuento del stock cuando un pedido se despacha.

No incluye:

- Contabilidad y facturación.
- Planillas de sueldos.
- Control de producción y fabricación.
- Lotes y fechas de vencimiento.
- Varias sucursales.
- Un portal para que las farmacias hagan sus pedidos por su cuenta.
- Pagos en línea.

Viabilidad del tema

Antes de seguir revisamos si el tema se puede trabajar como proyecto de la materia.

Pregunta	Respuesta
¿Existe una necesidad concreta?	Sí. Los pedidos se anotan hoy en cuadernos, hojas de Excel y mensajes, y eso genera pérdidas de información y confusión sobre el estado de cada pedido.
¿Hay usuarios identificables?	Sí. El ejecutivo de ventas que recibe los pedidos, el encargado de almacén que los prepara y el administrador que supervisa la información.

Pregunta	Respuesta
¿Se manejarán datos?	Sí. Farmacias, medicamentos con su precio y stock, pedidos con su detalle y usuarios con su rol.
¿Se puede construir en la materia?	Sí. Son pocas entidades y un solo proceso, con operaciones de registrar, consultar, modificar y desactivar que se pueden implementar por actividades.
¿Se podrá demostrar?	Sí. Con datos de prueba se puede registrar un pedido, despacharlo y ver cómo cambian su estado y el stock.

Proceso actual

Hoy el pedido empieza cuando una farmacia se comunica con el ejecutivo de ventas, casi siempre por llamada telefónica o por WhatsApp, y le dice qué medicamentos necesita y en qué cantidad. El ejecutivo de ventas es quien toma el pedido y el único que conoce la lista completa de lo que se pidió.

El ejecutivo anota el pedido en un cuaderno. Luego llama al almacén para preguntar si hay stock suficiente de cada medicamento, y con la respuesta confirma o ajusta lo que le dijo a la farmacia. Después copia el pedido a una hoja de Excel. El personal del almacén prepara los medicamentos a partir de lo que le avisan y despacha el pedido. Cuando la farmacia quiere saber cómo va su pedido, vuelve a llamar al ejecutivo, que tiene que buscar en el cuaderno o preguntar al almacén.

La información queda repartida en varios lugares: el cuaderno del ejecutivo, las hojas de Excel, los mensajes del chat y la memoria de las personas. Ninguno de esos lugares tiene el cuadro completo de qué se pidió, qué se despachó y qué sigue pendiente.

Problemas identificados

Del proceso anterior se desprenden varios problemas que se pueden observar sin necesidad de medirlos:

- Hay pedidos que se pierden o se anotan dos veces, porque dependen de un cuaderno y de mensajes sueltos.
- Al tomar el pedido no se conoce el stock real, así que a veces se promete algo que el almacén no tiene.
- Es difícil saber en qué estado está un pedido sin llamar a alguien.
- El almacén se entera tarde de los pedidos, porque el aviso llega por llamada o mensaje.
- No existe un historial ordenado de lo que ha pedido cada farmacia.
- Armar un reporte sencillo, por ejemplo los pedidos de una farmacia en un periodo, lleva mucho tiempo porque hay que revisar varias fuentes.

Descripción del problema

Laboratorio Químico Delta vende medicamentos al por mayor a las farmacias de Cochabamba. Hoy sus pedidos se reciben por teléfono o por mensajes, se anotan en un cuaderno y se copian a hojas de Excel, mientras el almacén se entera por llamadas. Por eso hay pedidos que se pierden o se repiten, el stock no se conoce al tomar el pedido y nadie sabe con certeza en qué estado está cada uno. Esto afecta al ejecutivo de ventas, al personal del almacén y a las farmacias, que esperan sin información clara. Se necesita una aplicación web que centralice farmacias, medicamentos y pedidos.

Justificación y objetivos

Justificación

El proceso que mejora este proyecto es el registro y el seguimiento de los pedidos mayoristas de las farmacias. Hoy esa información está repartida entre un cuaderno, hojas de Excel y mensajes; con el sistema pasaría a estar en un solo lugar, con un mismo formato para todos los pedidos.

Los beneficios son los que se pueden comprobar al probar la aplicación. Cada pedido queda registrado con su farmacia, sus medicamentos, sus cantidades y su total calculado por el sistema. El ejecutivo de ventas y el encargado de almacén ven el mismo estado del pedido (pendiente, despachado, entregado o anulado), sin tener que llamarse para preguntarlo. El stock de cada medicamento baja cuando el pedido se despacha, y el sistema avisa si no alcanza. Además, cada farmacia conserva su historial de pedidos y el administrador puede generar un reporte de los pedidos de una farmacia en un rango de fechas. La información que se ordena es el catálogo de medicamentos con su precio y stock, los datos de las farmacias y el detalle de cada pedido.

Objetivo general

Desarrollar una aplicación web para el registro y seguimiento de los pedidos mayoristas de farmacias de Laboratorio Químico Delta, que centralice la información de farmacias, medicamentos y pedidos.

Objetivos específicos

1. Analizar el proceso actual de atención de pedidos y definir los actores y los requerimientos funcionales y no funcionales, documentados en tablas de requerimientos.
2. Diseñar la arquitectura y el modelo de datos del sistema mediante casos de uso, diagramas C4 y modelos conceptual, lógico y físico, junto con el script SQL inicial.
3. Implementar los módulos de farmacias, medicamentos y pedidos bajo el patrón MVC, con operaciones de registro, consulta, modificación y desactivación.
4. Validar el funcionamiento del sistema con casos de prueba que verifiquen el registro de pedidos, el cambio de estado y el descuento de stock.

Usuarios y requerimientos

Actores del sistema

Identificamos tres actores, cada uno asociado a un rol dentro del proceso de pedidos.

Actor	Responsabilidad	Ejemplos de acciones
Administrador	Configura y supervisa la información del sistema.	Registrar usuarios y asignarles un rol, registrar medicamentos y categorías, actualizar precios, consultar pedidos y el reporte de pedidos.
Ejecutivo de ventas	Atiende a las farmacias y registra sus pedidos.	Registrar y actualizar farmacias, registrar pedidos, consultar el estado de un pedido, anular un pedido pendiente.
Encargado de almacén	Controla el stock y la salida de los pedidos.	Consultar el stock, marcar pedidos como despachados y como entregados.

No incluimos sistemas externos porque el alcance no tiene integración con correo, pagos ni facturación, y por eso el sistema no intercambia información con ningún otro. Tampoco agregamos más roles de los que el proceso necesita: con estos tres se cubre todo lo que ocurre desde que se registra una farmacia hasta que se entrega un pedido.

Requerimientos funcionales

La columna Actividad prevista indica qué requerimientos se construyen en la Actividad 2 (CRUD de farmacias y medicamentos) y cuáles quedan para actividades posteriores. Las categorías iniciales se cargan con el script SQL, por eso RF-13 se puede construir antes que RF-12.

Código	Requerimiento funcional	Actor	Prioridad	Actividad prevista
RF-01	El sistema deberá permitir iniciar sesión con correo y contraseña.	Administrador, Ejecutivo de ventas, Encargado de almacén	Alta	Act. 3
RF-02	El sistema deberá permitir cerrar la sesión.	Administrador, Ejecutivo de ventas, Encargado de almacén	Media	Act. 3
RF-03	El sistema deberá mostrar un panel principal con accesos a los módulos permitidos para el rol del usuario.	Administrador, Ejecutivo de ventas, Encargado de almacén	Media	Act. 3
RF-04	El sistema deberá permitir registrar un usuario con nombre completo, correo, contraseña y rol.	Administrador	Media	Act. 3
RF-05	El sistema deberá permitir desactivar un usuario para impedir su acceso.	Administrador	Media	Act. 3
RF-06	El sistema deberá permitir registrar una farmacia con nombre, NIT, responsable, teléfono, dirección y zona.	Ejecutivo de ventas	Alta	Act. 2
RF-07	El sistema deberá mostrar el listado de farmacias activas.	Ejecutivo de ventas	Alta	Act. 2
RF-08	El sistema deberá mostrar el detalle de una farmacia.	Ejecutivo de ventas	Alta	Act. 2
RF-09	El sistema deberá permitir modificar los datos de una farmacia.	Ejecutivo de ventas	Alta	Act. 2
RF-10	El sistema deberá permitir desactivar una farmacia conservando su historial de pedidos.	Ejecutivo de ventas	Alta	Act. 2
RF-11	El sistema deberá permitir buscar farmacias por nombre o NIT.	Ejecutivo de ventas	Media	Act. 3
RF-12	El sistema deberá permitir registrar una categoría de medicamentos con nombre y descripción.	Administrador	Media	Act. 3
RF-13	El sistema deberá permitir registrar un medicamento con nombre, presentación, categoría, precio mayorista y stock.	Administrador	Alta	Act. 2
RF-14	El sistema deberá mostrar el listado de medicamentos activos con su categoría, precio mayorista y stock.	Administrador, Ejecutivo de ventas, Encargado de almacén	Alta	Act. 2
RF-15	El sistema deberá mostrar el detalle de un medicamento.	Administrador, Ejecutivo de ventas, Encargado de almacén	Alta	Act. 2
RF-16	El sistema deberá permitir modificar los datos de un medicamento.	Administrador	Alta	Act. 2

Código	Requerimiento funcional	Actor	Prioridad	Actividad prevista
RF-17	El sistema deberá permitir desactivar un medicamento.	Administrador	Alta	Act. 2
RF-18	El sistema deberá permitir registrar un pedido para una farmacia activa con uno o más medicamentos y sus cantidades.	Ejecutivo de ventas	Alta	Act. 3
RF-19	El sistema deberá calcular automáticamente el subtotal de cada medicamento y el total del pedido.	Ejecutivo de ventas	Alta	Act. 3
RF-20	El sistema deberá mostrar el listado de pedidos filtrado por estado o por farmacia.	Administrador, Ejecutivo de ventas, Encargado de almacén	Alta	Act. 3
RF-21	El sistema deberá mostrar el detalle de un pedido con sus medicamentos, cantidades, precios y total.	Administrador, Ejecutivo de ventas, Encargado de almacén	Alta	Act. 3
RF-22	El sistema deberá permitir marcar un pedido pendiente como despachado.	Encargado de almacén	Alta	Act. 3
RF-23	El sistema deberá descontar del stock las cantidades de un pedido cuando se marca como despachado.	Encargado de almacén	Alta	Act. 3
RF-24	El sistema deberá impedir el despacho de un pedido si el stock de algún medicamento no alcanza, indicando cuál.	Encargado de almacén	Alta	Act. 3
RF-25	El sistema deberá permitir marcar un pedido despachado como entregado.	Encargado de almacén	Alta	Act. 3
RF-26	El sistema deberá permitir anular un pedido que todavía está pendiente.	Ejecutivo de ventas	Media	Act. 3
RF-27	El sistema deberá generar un reporte de pedidos por farmacia en un rango de fechas.	Administrador	Media	Act. 3

Requerimientos no funcionales

Los requerimientos no funcionales están escritos como condiciones que se pueden comprobar durante las pruebas.

Código	Tipo	Requerimiento no funcional	Justificación
RNF-01	Seguridad	La aplicación deberá pedir inicio de sesión antes de mostrar cualquier módulo y mostrar a cada usuario solo las opciones de su rol.	Los pedidos, los precios y los datos de las farmacias son información interna del laboratorio.
RNF-02	Seguridad	La aplicación deberá guardar las contraseñas como hash (bcrypt), de modo que en la tabla usuario no se vea la contraseña original.	Si alguien accede a la base de datos no obtiene las contraseñas de los usuarios.
RNF-03	Seguridad	La aplicación deberá ejecutar las consultas a la base de datos con parámetros, sin concatenar lo que escribe el usuario.	Evita la inyección SQL.
RNF-04	Compatibilidad	La aplicación deberá funcionar en las versiones actuales de Chrome, Edge y Firefox.	Son los navegadores que usa el personal del laboratorio.
RNF-05	Usabilidad	La aplicación deberá mostrar mensajes de validación y de error en español, que indiquen qué campo corregir y sin detalles técnicos como errores de SQL.	El usuario corrige el dato sin ayuda y no se expone información interna.
RNF-06	Usabilidad	La aplicación deberá poder usarse en computadora y en tablet, desde 768 píxeles de ancho, sin desplazamiento horizontal.	El personal del almacén puede trabajar con una tablet.

Código	Tipo	Requerimiento no funcional	Justificación
RNF-07	Rendimiento	La aplicación deberá responder en menos de 3 segundos en los listados y formularios, con los datos de prueba del proyecto y en el equipo de desarrollo.	Es una condición que se puede medir durante las pruebas.
RNF-08	Mantenibilidad	La aplicación deberá tener el código organizado con el patrón MVC, con modelos, vistas, controladores y rutas en carpetas separadas.	Facilita hacer cambios en las siguientes actividades.
RNF-09	Respaldo y versionado	El código y la documentación deberán estar versionados en GitHub, con commits descriptivos.	Evita la pérdida de trabajo y deja un historial de cambios.
RNF-10	Integridad	La aplicación deberá desactivar farmacias, medicamentos y usuarios mediante el campo activo, en lugar de eliminarlos.	Conserva el historial de pedidos.

Modelado UML

Para modelar los requerimientos usamos dos herramientas que se complementan: un diagrama de casos de uso y un conjunto de historias de usuario. El diagrama muestra qué actor usa cada función del sistema, y las historias agregan los criterios de aceptación, es decir, cuándo damos por cumplida cada necesidad. La guía permite presentar ambas cosas, así que repartimos la información entre las dos para no repetirla.

Diagrama de casos de uso

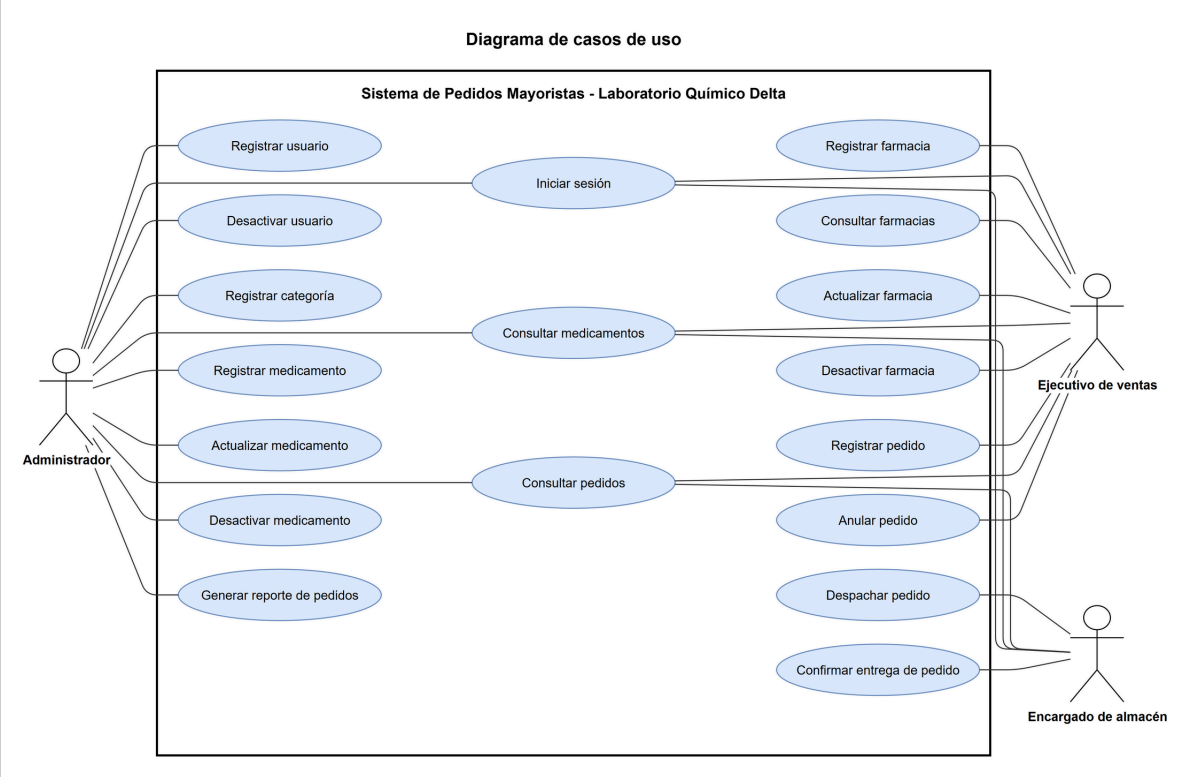


Diagrama de casos de uso

El rectángulo grande es el límite del sistema y lleva el nombre de la aplicación. Dentro están los dieciocho casos de uso, escritos con un verbo y un objeto, y fuera quedan los tres actores: el Administrador, el Ejecutivo de ventas y el Encargado de almacén. Cada línea une a un actor con un caso de uso que realiza. Iniciar sesión, Consultar medicamentos y Consultar pedidos son los casos que comparten los tres actores, por eso quedan en la columna del centro.

Los casos de uso se agrupan en los módulos que usamos en toda la documentación. Autenticación tiene Iniciar sesión; Usuarios tiene el registro y la desactivación de usuarios; Farmacias cubre el registro, la consulta, la actualización y la desactivación de farmacias; Medicamentos agrupa las categorías y el ciclo de vida de los medicamentos; y Pedidos reúne el registro, la consulta, el despacho, la entrega, la anulación y el reporte. El descuento de stock no tiene un óvalo propio porque ocurre dentro de Despachar pedido: al despachar, el sistema resta del stock las cantidades del pedido.

Casos de uso y requerimientos

Código	Caso de uso	Actor(es)	Requerimientos
CU-01	Iniciar sesión	Administrador, Ejecutivo de ventas, Encargado de almacén	RF-01, RF-02, RF-03
CU-02	Registrar usuario	Administrador	RF-04
CU-03	Desactivar usuario	Administrador	RF-05
CU-04	Registrar farmacia	Ejecutivo de ventas	RF-06
CU-05	Consultar farmacias	Ejecutivo de ventas	RF-07, RF-08, RF-11
CU-06	Actualizar farmacia	Ejecutivo de ventas	RF-09
CU-07	Desactivar farmacia	Ejecutivo de ventas	RF-10
CU-08	Registrar categoría	Administrador	RF-12
CU-09	Registrar medicamento	Administrador	RF-13
CU-10	Consultar medicamentos	Administrador, Ejecutivo de ventas, Encargado de almacén	RF-14, RF-15
CU-11	Actualizar medicamento	Administrador	RF-16
CU-12	Desactivar medicamento	Administrador	RF-17
CU-13	Registrar pedido	Ejecutivo de ventas	RF-18, RF-19
CU-14	Consultar pedidos	Administrador, Ejecutivo de ventas, Encargado de almacén	RF-20, RF-21
CU-15	Despachar pedido	Encargado de almacén	RF-22, RF-23, RF-24
CU-16	Confirmar entrega de pedido	Encargado de almacén	RF-25
CU-17	Anular pedido	Ejecutivo de ventas	RF-26
CU-18	Generar reporte de pedidos	Administrador	RF-27

Historias de usuario

Las catorce historias siguientes cubren los mismos requerimientos desde el punto de vista de quien los usa, con condiciones que se pueden comprobar al probar el sistema.

Código	Historia de usuario	Criterios de aceptación	Requerimientos
HU-01	Como usuario del sistema, quiero iniciar y cerrar sesión con mi correo y mi contraseña, para ver solo las opciones que corresponden a mi rol.	Con credenciales correctas entra al panel principal; con credenciales incorrectas ve un mensaje de error y no entra; al cerrar sesión no puede volver a los módulos sin ingresar otra vez.	RF-01, RF-02, RF-03
HU-02	Como Administrador, quiero registrar usuarios con su rol y desactivarlos, para controlar quién puede entrar al sistema.	El usuario se guarda con nombre completo, correo, contraseña y rol; el correo no puede repetirse; un usuario desactivado no puede iniciar sesión.	RF-04, RF-05
HU-03	Como Ejecutivo de ventas, quiero registrar una farmacia, para poder tomarle pedidos.	Se exigen nombre, NIT, teléfono, dirección y zona; el NIT no puede repetirse; la farmacia nueva aparece en el listado de activas.	RF-06
HU-04	Como Ejecutivo de ventas, quiero ver el listado de farmacias activas, buscar por nombre o NIT y abrir el detalle de una, para encontrar rápido a la farmacia que me llama.	El listado muestra solo farmacias activas; la búsqueda devuelve las que coinciden con el nombre o el NIT; el detalle muestra todos los datos de la farmacia.	RF-07, RF-08, RF-11
HU-05	Como Ejecutivo de ventas, quiero modificar o desactivar una farmacia, para mantener sus datos al día sin perder su historial.	Los cambios se ven en el detalle; el NIT modificado tampoco puede repetirse; una farmacia desactivada sale del listado de activas pero sus pedidos se conservan.	RF-09, RF-10
HU-06	Como Administrador, quiero registrar categorías y medicamentos, para mantener el catálogo que se ofrece a las farmacias.	El medicamento exige nombre, presentación, categoría, precio mayorista mayor que cero y stock igual o mayor que cero; el medicamento nuevo aparece en el listado; la categoría se guarda con nombre y descripción.	RF-12, RF-13
HU-07	Como Administrador, quiero modificar o desactivar un medicamento, para corregir precios y retirar los que ya no se venden.	El nuevo precio se muestra en el listado y no puede ser cero ni negativo; un medicamento desactivado sale del listado de activos; los pedidos anteriores que lo incluyen se conservan.	RF-16, RF-17
HU-08	Como Encargado de almacén, quiero ver los medicamentos con su stock y abrir el detalle de uno, para saber qué hay disponible antes de preparar un pedido.	El listado muestra categoría, precio mayorista y stock de cada medicamento activo; el detalle muestra todos sus datos.	RF-14, RF-15
HU-09	Como Ejecutivo de ventas, quiero registrar un pedido con varios medicamentos para una farmacia, para que el almacén sepa exactamente qué preparar.	Solo se puede elegir una farmacia activa; el pedido necesita al menos un medicamento y cada cantidad es mayor que cero; el sistema calcula subtotales y total; el pedido queda en estado pendiente.	RF-18, RF-19
HU-10	Como Ejecutivo de ventas o Encargado de almacén, quiero listar los pedidos por estado o por farmacia y abrir uno, para saber en qué punto está cada pedido.	El filtro por estado muestra solo los pedidos de ese estado; el filtro por farmacia muestra solo los de esa farmacia; el detalle muestra medicamentos, cantidades, precios y total.	RF-20, RF-21
HU-11	Como Ejecutivo de ventas, quiero anular un pedido que todavía está pendiente, para corregir un pedido tomado por error.	Un pedido pendiente pasa a anulado; un pedido despachado o entregado no se puede anular; el stock no cambia.	RF-26
HU-12	Como Encargado de almacén, quiero marcar un pedido pendiente como despachado, para que el stock refleje lo que salió del almacén.	Solo se despachan pedidos pendientes; al despachar baja el stock de cada medicamento en la cantidad pedida; si el stock de alguno no alcanza, el despacho se rechaza y el mensaje indica cuál medicamento falta.	RF-22, RF-23, RF-24
HU-13	Como Encargado de almacén, quiero marcar un pedido despachado como entregado, para cerrar el ciclo del pedido.	Solo un pedido despachado puede pasar a entregado; el pedido entregado queda en el historial de su farmacia.	RF-25

Código	Historia de usuario	Criterios de aceptación	Requerimientos
HU-14	Como Administrador, quiero generar el reporte de pedidos de una farmacia en un rango de fechas, para revisar lo que ha pedido en un periodo.	El reporte lista solo los pedidos de la farmacia elegida dentro de las fechas indicadas; muestra el estado y el total de cada pedido; si no hay pedidos, lo indica con un mensaje.	RF-27

Modelado C4

El modelo C4 describe una solución desde la vista más general hasta sus partes internas, en niveles que se acercan cada vez más. Aquí presentamos los niveles 1 a 3: el contexto, los contenedores y los componentes. El nivel 4, que baja hasta el código, lo dejamos para la implementación, como permite la guía.

Nivel 1: diagrama de contexto

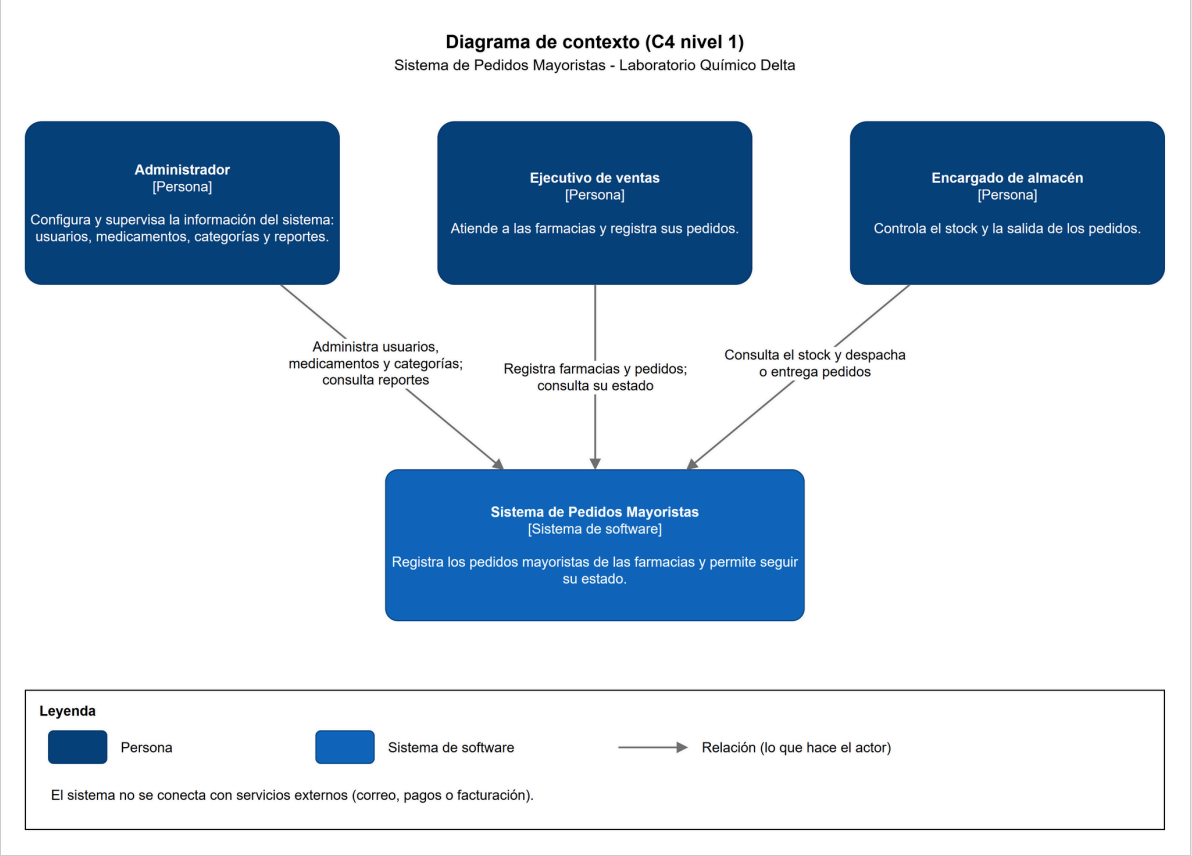


Diagrama de contexto C4

El sistema lo usan tres personas, cada una con una función distinta. El Administrador mantiene los usuarios, los medicamentos y las categorías y consulta los reportes; el Ejecutivo de ventas registra farmacias y pedidos y revisa su estado; el Encargado de almacén consulta el stock y despacha o entrega los pedidos. No dibujamos sistemas externos porque el alcance no tiene correo, pagos ni facturación, y no queríamos inventar integraciones que no existen.

Nivel 2: diagrama de contenedores

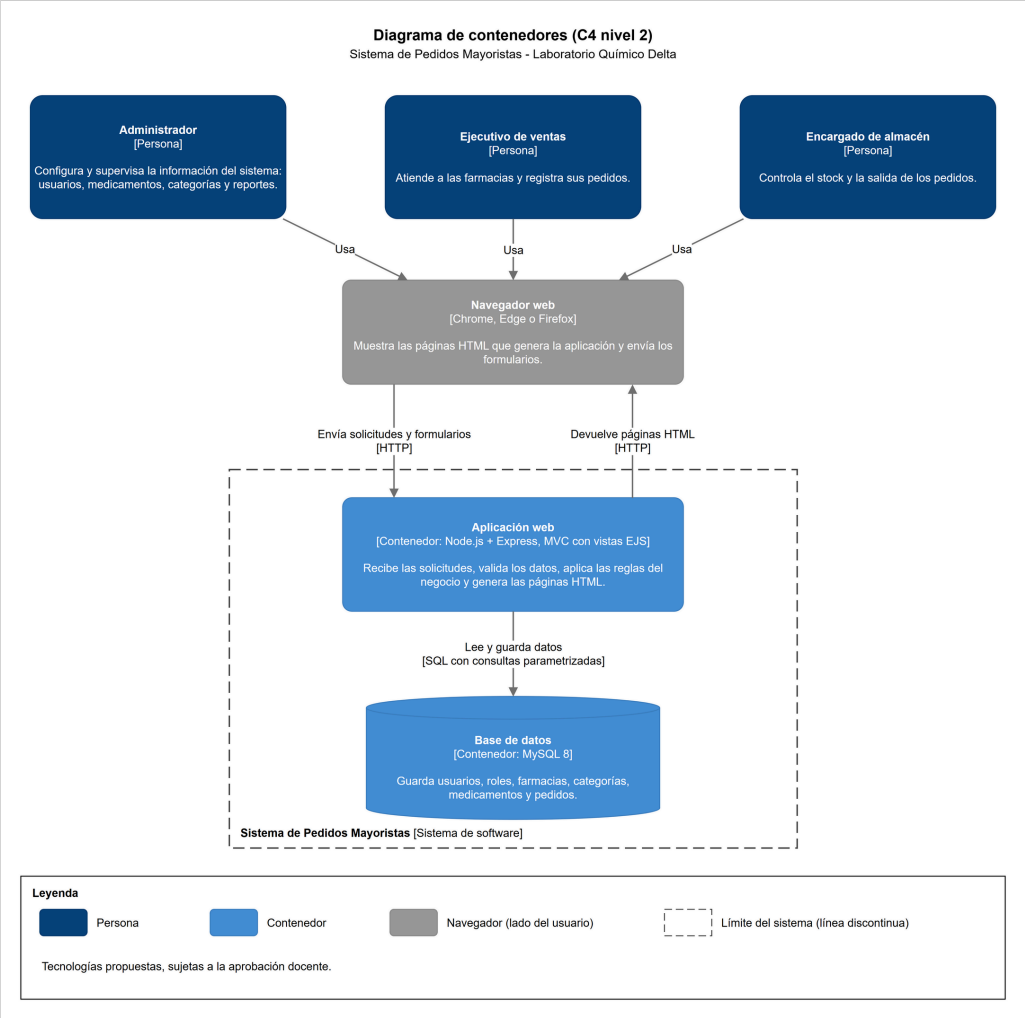


Diagrama de contenedores C4

Las personas trabajan desde un navegador web, que muestra las páginas y envía los formularios. Del otro lado está la Aplicación web, una sola aplicación Node.js con Express organizada con el patrón MVC, cuyas vistas EJS se generan en el servidor. Por eso no hay un frontend y un backend como proyectos separados: el mismo programa recibe la solicitud, aplica las reglas y devuelve la página ya armada. Para un sistema de este tamaño y un solo desarrollador nos pareció lo más simple de construir y de explicar. El tercer contenedor es la base de datos MySQL 8, donde se guarda toda la información.

La comunicación es sencilla: el navegador y la aplicación se hablan por HTTP, y la aplicación lee y guarda datos con consultas SQL parametrizadas. Un contenedor en C4 es una unidad de software o de almacenamiento que se ejecuta por separado, no un contenedor de Docker. Las tecnologías que aparecen en este nivel son una propuesta y están sujetas a la aprobación docente.

Nivel 3: diagrama de componentes

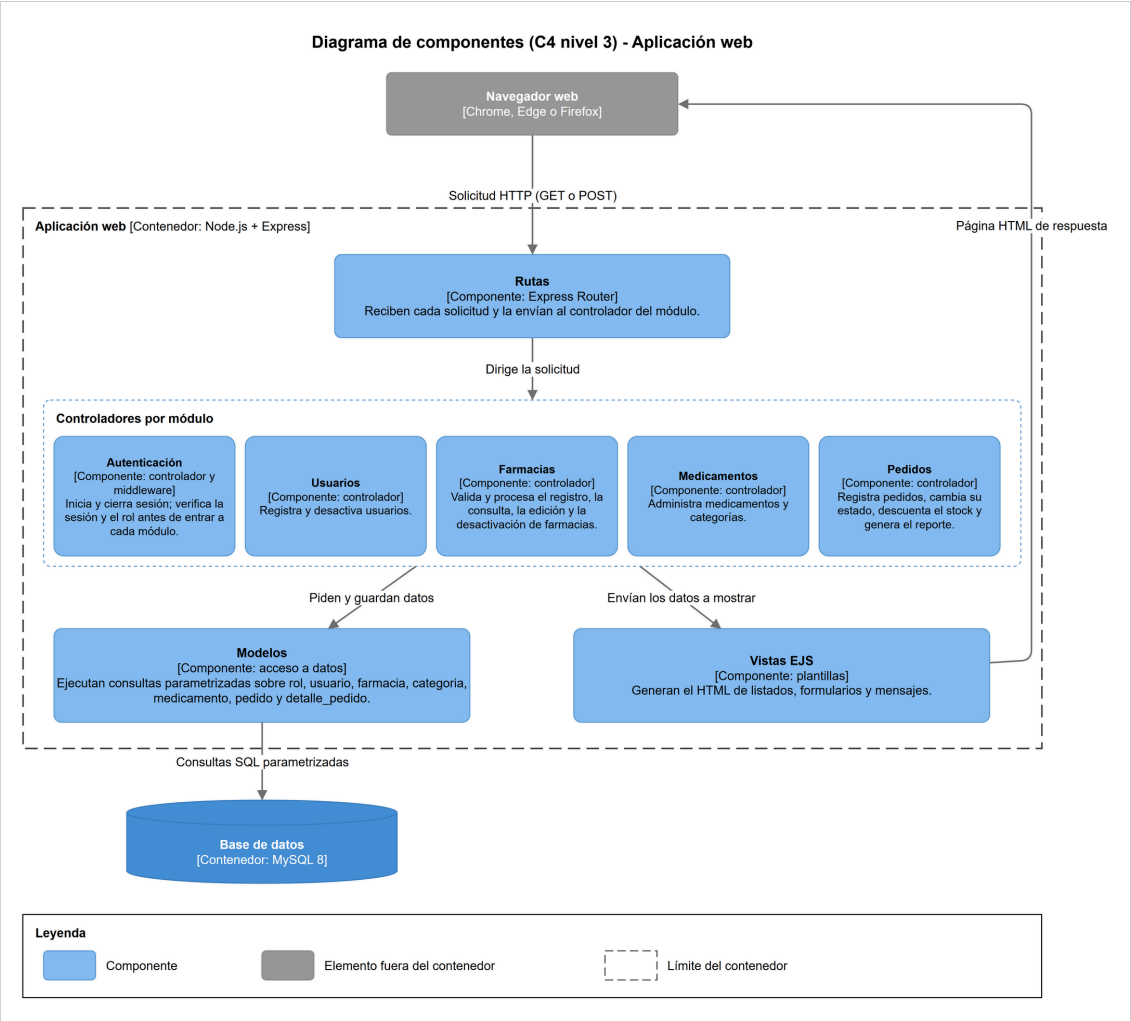


Diagrama de componentes C4

Aquí abrimos la Aplicación web. Las Rutas reciben cada solicitud que llega del navegador y la envían al controlador del módulo que corresponde. Antes de entrar a un módulo, el componente de Autenticación comprueba la sesión y el rol del usuario. Cada controlador (Usuarios, Farmacias, Medicamentos y Pedidos) valida los datos y aplica las reglas de su módulo. Los Modelos son los únicos que hablan con la base de datos, siempre con consultas parametrizadas, y las Vistas EJS arman el HTML de listados, formularios y mensajes que vuelve al navegador.

Arquitectura propuesta

Para ver cómo trabajan juntas las partes, seguimos una operación completa: el registro de una farmacia. El Ejecutivo de ventas abre en el navegador el formulario en la ruta GET /farmacias/nueva, lo llena y lo envía con POST /farmacias. Las Rutas dirigen la solicitud al controlador de Farmacias, después de que Autenticación confirma que hay una sesión iniciada con un rol que puede registrar farmacias. El controlador revisa que lleguen los campos obligatorios (nombre, NIT, teléfono, dirección y zona) y le pasa los datos al modelo de Farmacia, que inserta la fila en la tabla farmacia de MySQL con una consulta parametrizada.

Si el NIT ya existe, la restricción UNIQUE de la base de datos rechaza el registro y el controlador vuelve a mostrar el formulario con un mensaje claro, sin detalles técnicos. Si todo sale bien, redirige a GET /farmacias, donde la vista EJS dibuja el listado de farmacias activas con un mensaje de éxito y la página HTML vuelve al navegador. Despachar un pedido sigue el mismo camino, con la diferencia de que el modelo de Pedidos también descuenta el stock de los medicamentos.

Coherencia entre los diagramas

Los mismos tres actores aparecen en el análisis, en los casos de uso y en el diagrama de contexto, y la base de datos que dibujamos en C4 es la misma que modelamos en la sección de base de datos. La tabla siguiente relaciona cada módulo con sus casos de uso y con las tablas que usa.

Módulo	Casos de uso	Tablas que usa
Autenticación	Iniciar sesión	usuario, rol
Usuarios	Registrar usuario, Desactivar usuario	usuario, rol
Farmacias	Registrar farmacia, Consultar farmacias, Actualizar farmacia, Desactivar farmacia	farmacia
Medicamentos	Registrar categoría, Registrar medicamento, Consultar medicamentos, Actualizar medicamento, Desactivar medicamento	medicamento, categoria
Pedidos	Registrar pedido, Consultar pedidos, Despachar pedido, Confirmar entrega de pedido, Anular pedido, Generar reporte de pedidos	pedido, detalle_pedido, medicamento, farmacia

Base de datos

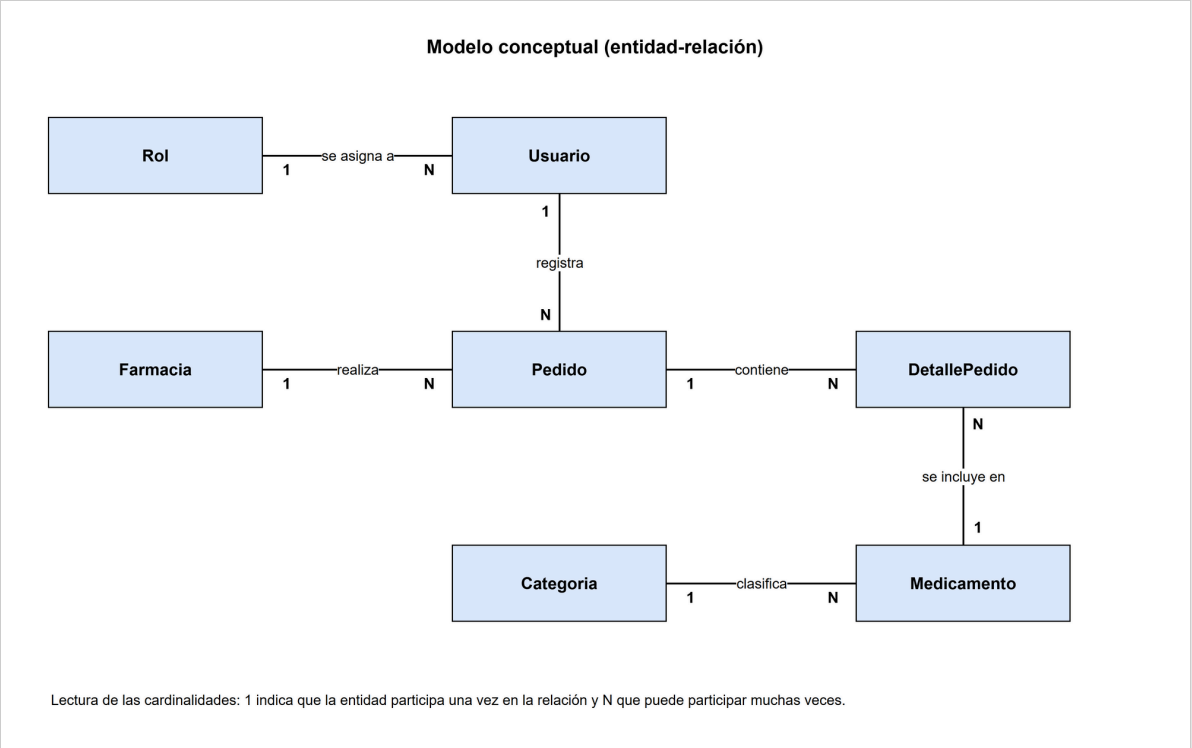
Para diseñar la base de datos partimos de los sustantivos que aparecen en los requerimientos funcionales: farmacia, medicamento, categoría, pedido, usuario y rol. De ahí salieron las entidades y las presentamos en tres modelos. El conceptual dice qué cosas existen y cómo se relacionan, sin pensar todavía en tablas. El lógico organiza esas cosas en tablas, campos y claves. El físico baja el lógico a un motor concreto, con tipos de dato y restricciones.

Entidades

Entidad	Qué información guarda	Ejemplos de atributos
Rol	Los tipos de usuario que existen en el sistema.	nombre, descripción
Usuario	Las personas que entran al sistema y el rol que tienen.	nombre completo, correo, clave, rol, activo
Farmacia	Las farmacias clientes del laboratorio.	nombre, NIT, responsable, teléfono, dirección, zona, activo
Categoria	Los grupos en que se clasifican los medicamentos.	nombre, descripción
Medicamento	El catálogo de medicamentos que se vende al por mayor.	nombre, presentación, categoría, precio mayorista, stock, activo
Pedido	Cada pedido que hace una farmacia.	farmacia, usuario que lo registró, fecha, estado, total, observación
DetallePedido	Cada línea de un pedido: un medicamento con su cantidad.	pedido, medicamento, cantidad, precio unitario, subtotal

DetallePedido existe porque un pedido lleva varios medicamentos y un medicamento aparece en muchos pedidos; sin esta entidad habría que repetir columnas dentro de Pedido.

Modelo conceptual

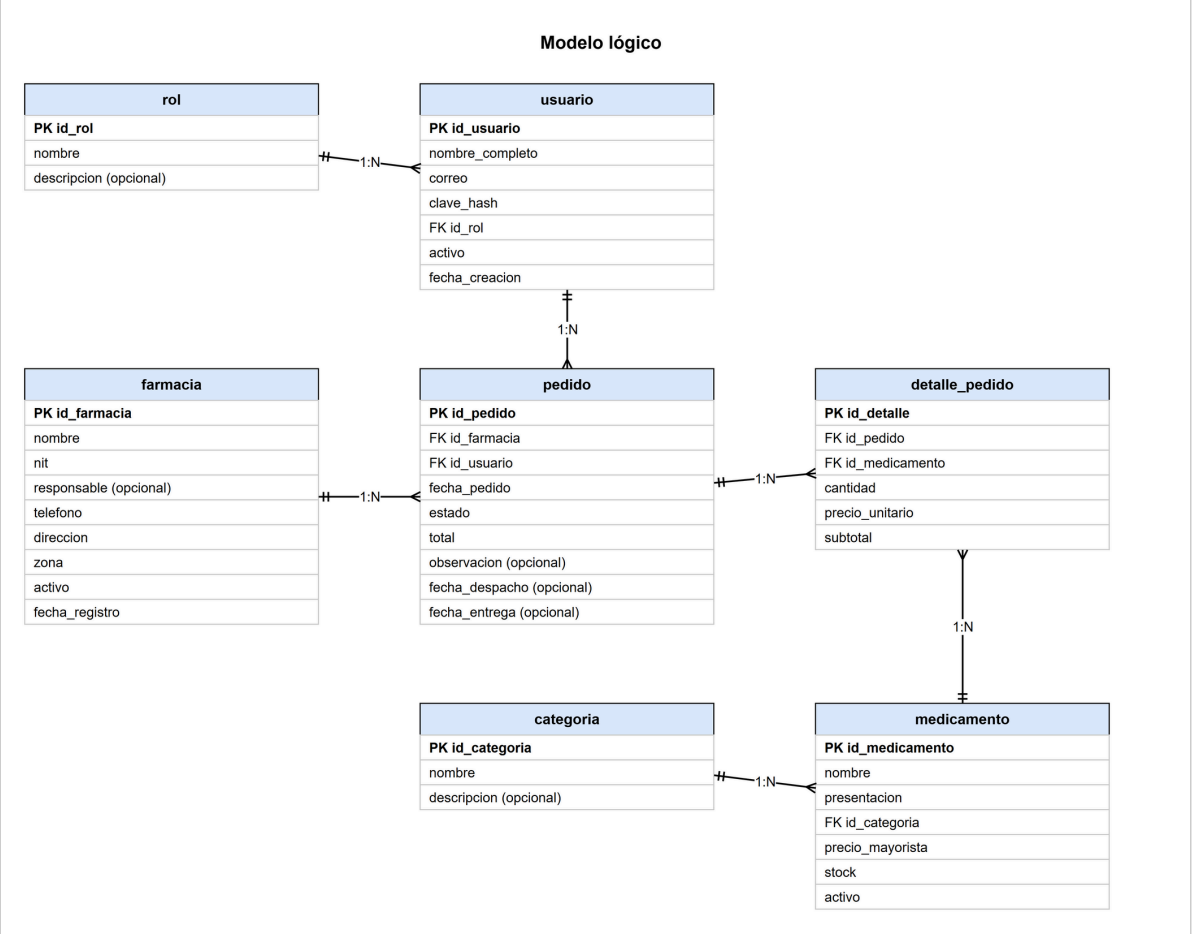


Modelo conceptual

Relación	Cardinalidad	Lectura
Rol se asigna a Usuario	1 : N	Un rol puede asignarse a muchos usuarios y cada usuario tiene un solo rol.
Usuario registra Pedido	1 : N	Un usuario puede registrar muchos pedidos y cada pedido lo registra un solo usuario.
Farmacia realiza Pedido	1 : N	Una farmacia puede realizar muchos pedidos y cada pedido pertenece a una sola farmacia.
Pedido contiene DetallePedido	1 : N	Un pedido contiene varias líneas y cada línea pertenece a un solo pedido.
Medicamento se incluye en DetallePedido	1 : N	Un medicamento puede aparecer en muchas líneas de pedido y cada línea se refiere a un solo medicamento.
Categoria clasifica Medicamento	1 : N	Una categoría agrupa muchos medicamentos y cada medicamento pertenece a una sola categoría.

Entre Pedido y Medicamento la relación real es de muchos a muchos, porque un pedido incluye varios medicamentos y un medicamento está en varios pedidos. DetallePedido la resuelve y además guarda el precio que tenía el medicamento al momento del pedido.

Modelo lógico



Modelo lógico

Tabla	Campos principales	Clave / relación
rol	id_rol, nombre, descripcion	PK id_rol
usuario	id_usuario, nombre_completo, correo, clave_hash, id_rol, activo, fecha_creacion	PK id_usuario; FK id_rol a rol
farmacia	id_farmacia, nombre, nit, responsable, telefono, direccion, zona, activo, fecha_registro	PK id_farmacia
categoria	id_categoria, nombre, descripcion	PK id_categoria
medicamento	id_medicamento, nombre, presentacion, id_categoria, precio_mayorista, stock, activo	PK id_medicamento; FK id_categoria a categoria
pedido	id_pedido, id_farmacia, id_usuario, fecha_pedido, estado, total, observacion, fecha_despacho, fecha_entrega	PK id_pedido; FK id_farmacia a farmacia; FK id_usuario a usuario
detalle_pedido	id_detalle, id_pedido, id_medicamento, cantidad, precio_unitario, subtotal	PK id_detalle; FK id_pedido a pedido; FK id_medicamento a medicamento

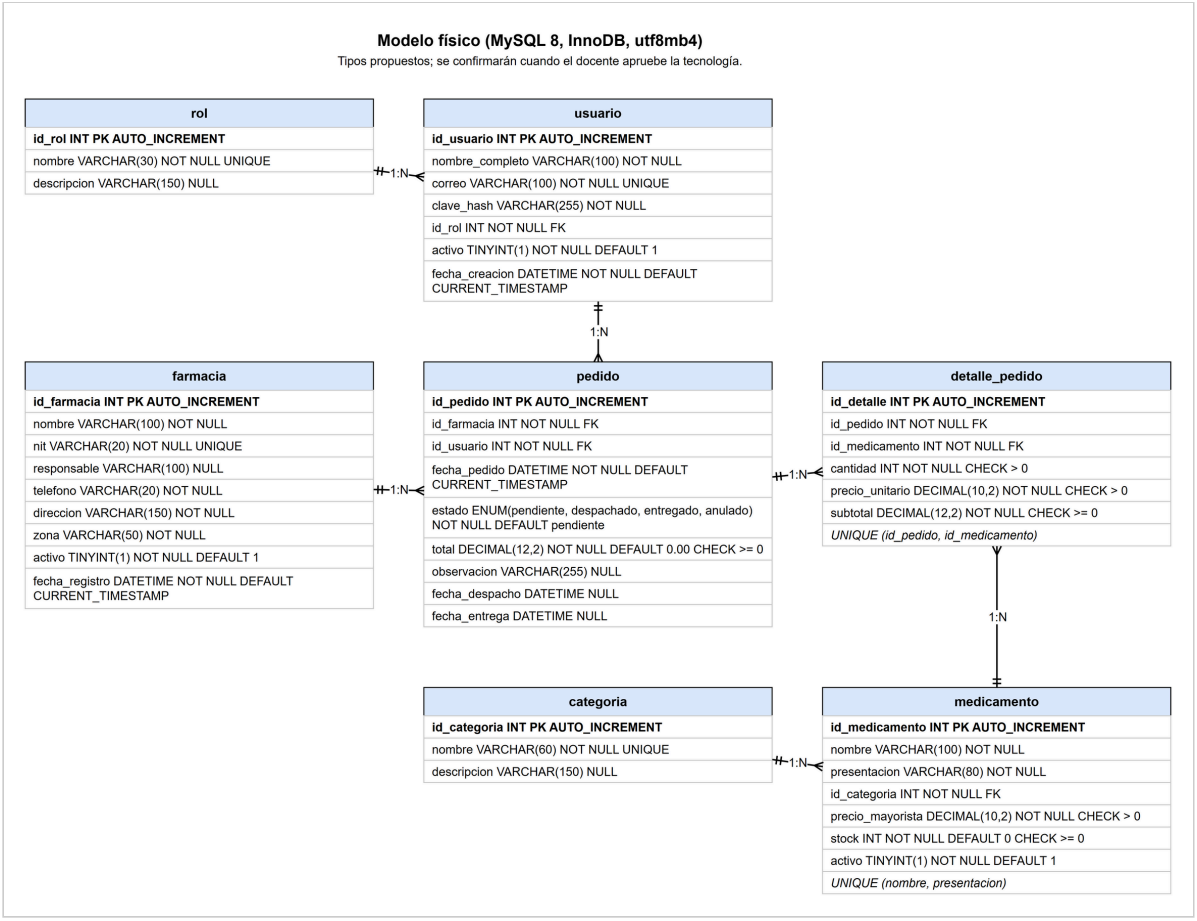
Los campos opcionales son la descripción de rol y categoría, el responsable de la farmacia, la observación del pedido y las fechas de despacho y entrega, que solo se llenan cuando el pedido avanza de estado.

Revisión del modelo

Revisamos el modelo lógico con las preguntas de la guía. Todas las tablas tienen una clave primaria, y cada clave foránea apunta a una tabla que existe. Un pedido guarda muchos medicamentos a través de detalle_pedido, sin columnas repetidas como medicamento1 o medicamento2. El nombre de la categoría no se repite en la tabla medicamento: allí solo se guarda id_categoria.

El precio unitario se copia a detalle_pedido porque el precio mayorista de un medicamento puede cambiar después, y los pedidos anteriores deben conservar el precio con el que se vendieron. El subtotal y el total los calcula el sistema al registrar el pedido y los guardamos para que las consultas y el reporte sean más simples. Por último, farmacia, medicamento y usuario tienen el campo activo, de modo que se desactivan en lugar de borrarse y el historial de pedidos se conserva (RNF-10).

Modelo físico



Modelo físico

Proponemos MySQL 8 con el motor InnoDB y el juego de caracteres utf8mb4, para que los acentos se guarden bien. Como la tecnología todavía debe aprobarla el docente, los tipos de dato se deben confirmar antes de implementar. Las reglas de integridad que incluimos son: campos obligatorios con NOT NULL, valores únicos en el NIT de la farmacia, el correo del usuario y los nombres de rol y categoría, restricciones CHECK para que el precio mayorista, el precio unitario y la cantidad sean mayores que cero y para que el stock, el subtotal y el total no sean negativos, y las seis claves foráneas entre tablas.

Diccionario de datos

Tabla / campo	Tipo propuesto	Regla	Descripción
rol.id_rol	INT	PK, automático	Identificador del rol.
rol.nombre	VARCHAR(30)	Obligatorio, único	Nombre del rol: Administrador, Ejecutivo de ventas o Encargado de almacén.
rol.descripcion	VARCHAR(150)	Opcional	Explicación breve de lo que puede hacer el rol.
usuario.id_usuario	INT	PK, automático	Identificador del usuario.
usuario.nombre_completo	VARCHAR(100)	Obligatorio	Nombre y apellido de la persona que usa el sistema.
usuario.correo	VARCHAR(100)	Obligatorio, único	Correo con el que inicia sesión.
usuario.clave_hash	VARCHAR(255)	Obligatorio	Contraseña guardada como hash, nunca en texto plano.
usuario.id_rol	INT	Obligatorio, FK a rol	Rol asignado al usuario.
usuario.activo	TINYINT(1)	Obligatorio, valor por defecto 1	Indica si el usuario puede entrar (1) o fue desactivado (0).
usuario.fecha_creacion	DATETIME	Obligatorio, valor por defecto la fecha actual	Fecha y hora en que se registró el usuario.
farmacia.id_farmacia	INT	PK, automático	Identificador de la farmacia.
farmacia.nombre	VARCHAR(100)	Obligatorio	Nombre comercial de la farmacia.
farmacia.nit	VARCHAR(20)	Obligatorio, único	Número de identificación tributaria de la farmacia.
farmacia.responsable	VARCHAR(100)	Opcional	Persona encargada de hacer los pedidos en la farmacia.
farmacia.telefono	VARCHAR(20)	Obligatorio	Teléfono de contacto.

Tabla / campo	Tipo propuesto	Regla	Descripción
farmacia.direccion	VARCHAR(150)	Obligatorio	Dirección donde se entregan los pedidos.
farmacia.zona	VARCHAR(50)	Obligatorio	Zona de la ciudad en que está la farmacia.
farmacia.activo	TINYINT(1)	Obligatorio, valor por defecto 1	Indica si la farmacia está activa (1) o desactivada (0).
farmacia.fecha_registro	DATETIME	Obligatorio, valor por defecto la fecha actual	Fecha y hora en que se registró la farmacia.
categoria.id_categoria	INT	PK, automático	Identificador de la categoría.
categoria.nombre	VARCHAR(60)	Obligatorio, único	Nombre de la categoría de medicamentos.
categoria.descripcion	VARCHAR(150)	Opcional	Descripción breve de la categoría.
medicamento.id_medicamento	INT	PK, automático	Identificador del medicamento.
medicamento.nombre	VARCHAR(100)	Obligatorio, único junto con presentacion	Nombre del medicamento con su concentración.
medicamento.presentacion	VARCHAR(80)	Obligatorio	Forma en que se vende, por ejemplo caja de tabletas.
medicamento.id_categoria	INT	Obligatorio, FK a categoria	Categoría a la que pertenece.
medicamento.precio_mayorista	DECIMAL(10,2)	Obligatorio, mayor que cero	Precio de venta al por mayor.
medicamento.stock	INT	Obligatorio, igual o mayor que cero, valor por defecto 0	Unidades disponibles en el almacén.
medicamento.activo	TINYINT(1)	Obligatorio, valor por defecto 1	Indica si el medicamento se sigue vendiendo (1) o fue desactivado (0).
pedido.id_pedido	INT	PK, automático	Identificador del pedido.

Tabla / campo	Tipo propuesto	Regla	Descripción
pedido.id_farmacia	INT	Obligatorio, FK a farmacia	Farmacia que hizo el pedido.
pedido.id_usuario	INT	Obligatorio, FK a usuario	Usuario que registró el pedido.
pedido.fecha_pedido	DATETIME	Obligatorio, valor por defecto la fecha actual	Fecha y hora en que se registró el pedido.
pedido.estado	ENUM('pendiente','despachado','entregado','anulado')	Obligatorio, valor por defecto pendiente	Estado actual del pedido.
pedido.total	DECIMAL(12,2)	Obligatorio, igual o mayor que cero, valor por defecto 0.00	Suma de los subtotales del pedido, calculada por el sistema.
pedido.observacion	VARCHAR(255)	Opcional	Nota libre sobre el pedido.
pedido.fecha_despacho	DATETIME	Opcional	Fecha y hora en que el almacén despachó el pedido.
pedido.fecha_entrega	DATETIME	Opcional	Fecha y hora en que se confirmó la entrega.
detalle_pedido.id_detalle	INT	PK, automático	Identificador de la línea del pedido.
detalle_pedido.id_pedido	INT	Obligatorio, FK a pedido, único junto con id_medicamento	Pedido al que pertenece la línea.
detalle_pedido.id_medicamento	INT	Obligatorio, FK a medicamento	Medicamento pedido en esa línea.
detalle_pedido.cantidad	INT	Obligatorio, mayor que cero	Unidades pedidas.
detalle_pedido.precio_unitario	DECIMAL(10,2)	Obligatorio, mayor que cero	Precio mayorista vigente cuando se registró el pedido.
detalle_pedido.subtotal	DECIMAL(12,2)	Obligatorio, igual o mayor que cero	Cantidad multiplicada por el precio unitario.

Script SQL inicial

El archivo database/script_inicial.sql es un borrador: crea la base de datos delta_pedidos, las siete tablas y algunos datos ficticios de prueba (roles, categorías, medicamentos y farmacias). Lo vamos a ejecutar cuando se instale el servidor MySQL en la siguiente actividad. No crea usuarios del sistema, porque las contraseñas deben guardarse como hash (RNF-02) y todavía no existe esa parte de la aplicación.

Prototipo de interfaces

Para revisar la navegación y los campos antes de programar elaboramos un prototipo navegable con páginas HTML y una hoja de estilos común. Elegimos HTML en lugar de Figma porque nos permitía armar las pantallas rápido y, además, reutilizar los mismos estilos en las vistas EJS de la aplicación real. El prototipo está en la carpeta prototipos/ del repositorio y se abre con cualquier navegador desde el archivo index.html. Las pantallas no guardan datos: los nombres, montos y mensajes que aparecen son de ejemplo y sirven solo para mostrar cómo se verá cada función.

Todas las pantallas internas comparten la misma estructura: una barra superior con el nombre del laboratorio y del usuario que inició sesión, un menú lateral con los módulos agrupados (ventas, catálogo y administración) y el área de contenido. Los botones principales van en azul, las acciones de desactivar o anular en rojo y los estados de los pedidos se muestran con etiquetas de color, para que se distingan de un vistazo.

Pantallas y requerimientos relacionados

Pantalla	Archivo	Requerimientos
Inicio de sesión	index.html	RF-01
Panel principal	panel.html	RF-02, RF-03
Listado de farmacias	farmacias.html	RF-07, RF-11
Registro y edición de farmacia	farmacia-form.html	RF-06, RF-09
Detalle de farmacia	farmacia-detalle.html	RF-08
Confirmación para desactivar farmacia	farmacia-desactivar.html	RF-10
Listado de medicamentos	medicamentos.html	RF-14, RF-17
Registro y edición de medicamento	medicamento-form.html	RF-13, RF-16
Detalle de medicamento	medicamento-detalle.html	RF-15
Listado de pedidos	pedidos.html	RF-20, RF-26
Registro de pedido	pedido-nuevo.html	RF-18, RF-19
Detalle de pedido	pedido-detalle.html	RF-21, RF-22, RF-23, RF-24, RF-25
Usuarios	usuarios.html	RF-04, RF-05
Reporte de pedidos por farmacia	reporte.html	RF-27
Registro no encontrado	no-encontrado.html	RNF de mensajes claros

El registro de categorías (RF-12) no tiene una pantalla propia en esta versión del prototipo, porque las categorías iniciales se cargan con el script de la base de datos; su formulario será igual de sencillo que el de farmacias y se agregará cuando se implemente.

Mensajes de éxito, validación y error

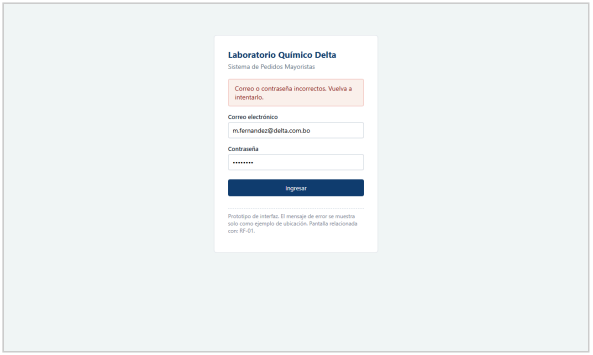
En el prototipo dejamos marcados los lugares donde aparecerán los mensajes. En el listado de farmacias se ve el mensaje de confirmación después de registrar una farmacia. En los formularios de farmacia y de medicamento se muestran errores de validación debajo de cada campo, por ejemplo un NIT repetido, un teléfono vacío o un precio negativo, además de un aviso general en la parte superior. En el detalle del pedido aparece el error que impide despachar cuando el stock de un medicamento no alcanza, y la pantalla de inicio de sesión muestra el mensaje para credenciales incorrectas.

Navegación

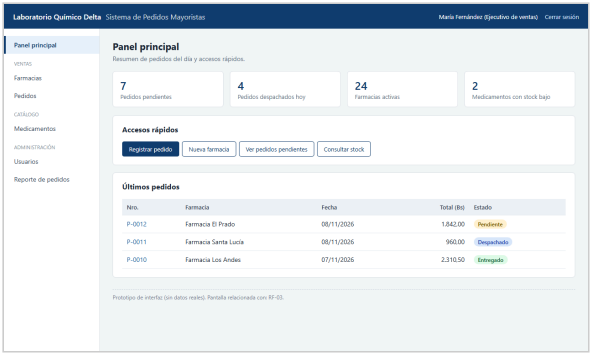
El recorrido principal empieza en el inicio de sesión, que lleva al panel. Desde el panel y desde el menú lateral se llega a cada módulo. En cada listado, el botón de nuevo registro abre el formulario, y los botones Guardar y Cancelar devuelven al listado. Desde el detalle de una farmacia se puede ir directamente a registrar un pedido para ella, y desde el listado de pedidos se abre el detalle donde el encargado de almacén cambia el estado.

Capturas del prototipo

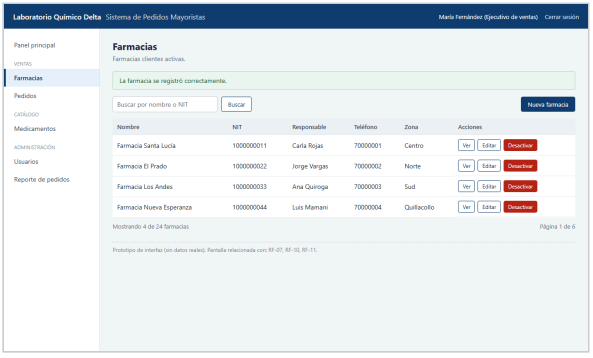
Incluimos las pantallas más representativas del recorrido. Las quince capturas completas están en la carpeta prototipos/capturas/ del repositorio y el prototipo se puede recorrer abriendo prototipos/index.html.



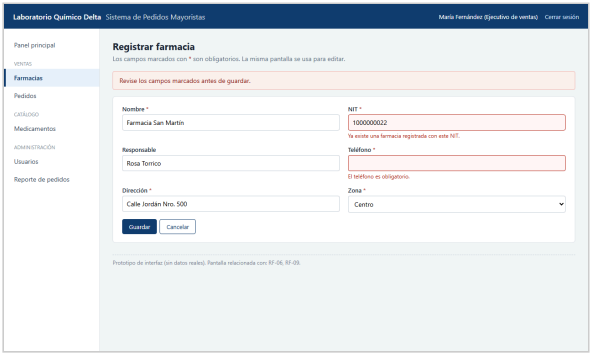
Inicio de sesión



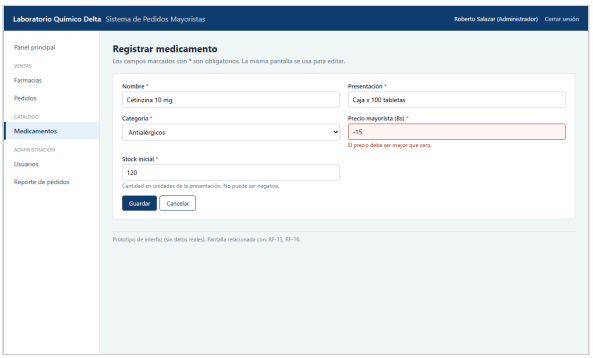
Panel principal



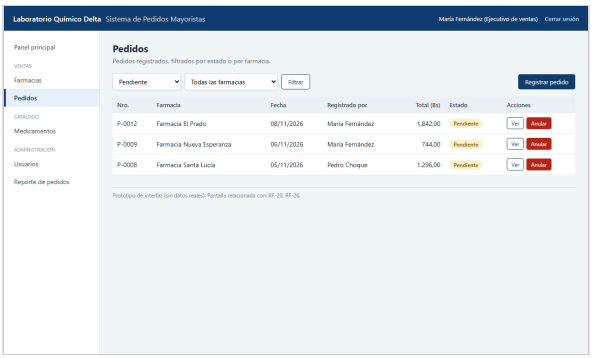
Listado de farmacias



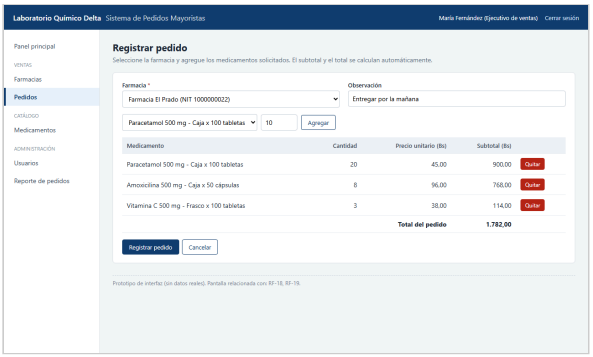
Formulario de farmacia con errores de validación



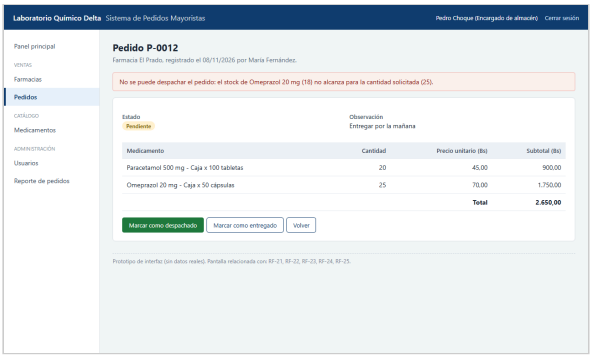
Formulario de medicamento



Listado de pedidos



Registro de pedido



Detalle de pedido con error de stock

Tecnologías seleccionadas

La selección la hicimos pensando en el alcance del proyecto, en lo que ya conocemos y en que cada herramienta permita organizar el sistema bajo el patrón MVC. Esta propuesta queda sujeta a la aprobación docente; si el docente define una tecnología común para el curso, ajustaremos la matriz antes de seguir con la implementación.

Matriz tecnológica

Parte	Tecnología propuesta	¿Para qué se utilizará?	Justificación
Backend	Node.js con Express 4	Recibir las solicitudes, aplicar las reglas del negocio y coordinar el acceso a los datos.	Express es liviano y deja organizar el código en rutas, controladores y modelos, que es justamente la separación que pide MVC. Ya trabajamos con JavaScript en otras materias y su documentación es amplia.
Frontend	Vistas EJS con HTML y CSS	Construir las pantallas que usan las personas, generadas desde el servidor.	Con EJS las vistas se arman en el mismo proyecto y reciben los datos del controlador, sin necesidad de una aplicación aparte. Reutiliza los estilos del prototipo.
Base de datos	MySQL 8 (motor InnoDB)	Guardar farmacias, medicamentos, pedidos y usuarios.	Es relacional, soporta claves foráneas, restricciones CHECK y transacciones, que necesitamos para registrar un pedido con su detalle y descontar stock. Lo usamos en Administración de Base de Datos.
Acceso a datos	Librería mysql2	Ejecutar las consultas desde los modelos.	Permite consultas con parámetros, lo que evita la inyección SQL, y maneja un grupo de conexiones reutilizables.

Parte	Tecnología propuesta	¿Para qué se utilizará?	Justificación
Gestor de base de datos	MySQL Workbench	Crear la base, ejecutar el script y revisar las tablas.	Es la herramienta oficial de MySQL y muestra la estructura de forma visual.
Editor	Visual Studio Code	Escribir el código y la documentación.	Es gratuito, tiene terminal integrada y extensiones para JavaScript, EJS y Git.
Diagramación	draw.io (diagrams.net)	Elaborar los diagramas UML, C4 y de base de datos.	Guarda archivos editables .drawio y exporta imágenes PNG para el documento.
Prototipado	HTML y CSS	Diseñar y revisar las pantallas antes de programar.	El prototipo se abre en cualquier navegador y sus estilos se reutilizan en la aplicación.
Control de versiones	Git y GitHub	Registrar los cambios y respaldar el código y la documentación.	Mantiene el historial del trabajo y permite que el docente revise el avance.
Pruebas de rutas	Navegador web y Postman	Comprobar cada operación y sus respuestas.	Las rutas se pueden probar desde los formularios y también enviando solicitudes directas.
Despliegue	Servicio en la nube con soporte para Node.js y MySQL (por ejemplo Railway o Render)	Publicar la aplicación para la Actividad 4.	Tienen planes gratuitos o de bajo costo y se conectan con el repositorio de GitHub. La elección final se hará en la Actividad 4.

Rutas previstas para la Actividad 2

Como la aplicación genera sus vistas desde el servidor, las operaciones se exponen como rutas MVC que reciben formularios. Los formularios HTML solo envían GET y POST, por eso la edición y la desactivación usan POST con una ruta propia en lugar de PUT y DELETE. En esta etapa dejamos identificadas las rutas de los dos módulos que se implementarán primero.

Recurso	Operación	Método y ruta
Farmacias	Listar farmacias activas (con búsqueda opcional)	GET /farmacias
Farmacias	Mostrar el formulario de registro	GET /farmacias/nueva
Farmacias	Registrar una farmacia	POST /farmacias
Farmacias	Ver el detalle	GET /farmacias/:id
Farmacias	Mostrar el formulario de edición	GET /farmacias/:id/editar
Farmacias	Guardar los cambios	POST /farmacias/:id/editar
Farmacias	Desactivar	POST /farmacias/:id/desactivar
Medicamentos	Listar medicamentos activos (filtro por categoría)	GET /medicamentos
Medicamentos	Mostrar el formulario de registro	GET /medicamentos/nuevo
Medicamentos	Registrar un medicamento	POST /medicamentos
Medicamentos	Ver el detalle	GET /medicamentos/:id
Medicamentos	Mostrar el formulario de edición	GET /medicamentos/:id/editar
Medicamentos	Guardar los cambios	POST /medicamentos/:id/editar
Medicamentos	Desactivar	POST /medicamentos/:id/desactivar

Las rutas de pedidos, usuarios y autenticación (por ejemplo /pedidos, /pedidos/:id/despachar y /login) se definirán en la Actividad 3 siguiendo la misma convención.

Estructura inicial del desarrollo

El repositorio se organiza en una sola aplicación con las carpetas de documentación del proyecto y las carpetas propias de MVC dentro de src/.

Carpeta o archivo	Contenido
documentacion/	Textos de cada actividad en Markdown y los documentos entregados.
diagramas/	Diagramas UML, C4 y de base de datos en .drawio y .png.
prototipos/	Prototipo navegable en HTML y sus capturas.
database/	Script SQL inicial de la base de datos.
src/config/	Configuración de la conexión a MySQL.
src/models/	Modelos: consultas y operaciones sobre cada tabla.
src/controllers/	Controladores de cada módulo.
src/routes/	Rutas que relacionan cada dirección con una acción del controlador.
src/validators/	Validaciones de los datos que llegan de los formularios.
src/views/	Vistas EJS organizadas por módulo.
public/	Archivos estáticos, como la hoja de estilos.
.env.example	Ejemplo de variables de entorno, sin contraseñas reales.

Planificación del desarrollo

Cronograma inicial

Dividimos el proyecto siguiendo las seis actividades de la materia. Las fechas son estimadas y las iremos ajustando según las fechas que se publiquen en la plataforma. El orden respeta las dependencias: primero el análisis y el modelo de datos, luego las tablas y el CRUD, después las interfaces y por último las pruebas y el despliegue.

Nro.	Tarea	Fecha estimada	Resultado esperado
1	Análisis del proceso, actores y requerimientos	Semana del 2 al 8 de noviembre	Problema, objetivos y tablas de RF y RNF.
2	Casos de uso, historias de usuario y modelo C4	Semana del 2 al 8 de noviembre	Diagramas en .drawio y .png.
3	Modelo de datos y script SQL inicial	Semana del 2 al 8 de noviembre	Modelos conceptual, lógico y físico, diccionario y script.
4	Prototipo de interfaces y repositorio	Semana del 2 al 8 de noviembre	Prototipo navegable y repositorio con README.
5	Estructura MVC y conexión a MySQL	Semana del 2 al 8 de noviembre	Aplicación que inicia y se conecta a la base.
6	CRUD de farmacias y medicamentos con validaciones	Semana del 2 al 8 de noviembre	Módulos funcionando y probados (Actividad 2).
7	Inicio de sesión, usuarios y roles	Semana del 9 al 15 de noviembre	Acceso restringido según el rol.
8	Registro de pedidos, cambio de estado y descuento de stock	Semana del 16 al 22 de noviembre	Ciclo completo de un pedido (Actividad 3).
9	Reporte de pedidos por farmacia	Semana del 16 al 22 de noviembre	Reporte por rango de fechas.
10	Plan de pruebas y corrección de errores	Semana del 23 al 29 de noviembre	Casos de prueba ejecutados y documentados.

Nro.	Tarea	Fecha estimada	Resultado esperado
11	Despliegue en la nube	Semana del 23 al 29 de noviembre	Aplicación publicada y accesible (Actividad 4).
12	Preparación del examen práctico integrador	Primera semana de diciembre	Sistema estable para la Actividad 5.
13	Ajustes finales y documentación completa	Segunda semana de diciembre	Proyecto terminado (Actividad 6).

Forma de organizar el trabajo

Como el proyecto es individual, usaremos un tablero Kanban personal en GitHub Projects, vinculado al mismo repositorio, con tres columnas: Pendiente, En proceso y Terminado. Cada tarea del cronograma se divide en tarjetas pequeñas que se puedan terminar en una o dos sesiones de trabajo, por ejemplo "formulario de registro de farmacia" o "validar NIT repetido". Al empezar una sesión se pasa una tarjeta a En proceso, y solo se mueve a Terminado cuando la función está probada y guardada con un commit.

Al final de cada semana revisaremos el tablero contra el cronograma. Si una tarea se atrasa, primero se reduce su alcance a lo esencial y lo demás se anota como pendiente para la siguiente actividad. Elegimos Kanban en lugar de Scrum porque no tenemos un equipo con quien hacer reuniones ni sprints fijos; nos basta con ver qué está pendiente y limitar el trabajo en curso a una o dos tareas.

Riesgos y acciones preventivas

Riesgo	Probabilidad / impacto	Acción preventiva
Pérdida de archivos o del código	Media / Alta	Hacer commits frecuentes y subir los cambios a GitHub al terminar cada sesión.
Alcance demasiado grande para el tiempo disponible	Alta / Alta	Mantener un solo proceso (pedidos) y dejar fuera lo que no aporte a ese proceso, como lotes o facturación.
Que el docente no apruebe la tecnología propuesta	Media / Alta	Presentar la matriz tecnológica en esta actividad y ajustar el proyecto apenas se reciba la observación.
Problemas de instalación o configuración de MySQL	Media / Media	Documentar los pasos de instalación y guardar el script SQL para recrear la base en cualquier equipo.
Exposición de contraseñas en el repositorio	Baja / Alta	Guardar las credenciales en un archivo .env excluido con .gitignore y subir solo un .env.example.
Dificultad para desplegar la aplicación	Media / Media	Probar el servicio de despliegue con una versión pequeña antes de la Actividad 4.
Retraso en las tareas por otras materias	Media / Alta	Dividir el trabajo en tarjetas pequeñas y revisar el tablero cada semana.

Repositorio GitHub

El proyecto se encuentra en el siguiente repositorio público:

https://github.com/GustavoRodVe/sistema-pedidos-delta

El repositorio tiene un archivo README con el nombre, el propósito, el alcance y las herramientas del proyecto, y está organizado en las carpetas documentacion/, diagramas/, prototipos/ y database/. En la carpeta de diagramas se guardan tanto los archivos editables .drawio como las imágenes

exportadas, para que cualquier diagrama se pueda corregir más adelante. Los cambios se fueron registrando con commits que describen lo que se agregó en cada paso, como la estructura inicial, los requerimientos, los diagramas, el modelo de datos y el prototipo. En el repositorio no se guardan contraseñas ni datos reales: la configuración de la base se lee de un archivo .env que está excluido con .gitignore.

Conclusiones

El análisis del proceso de pedidos nos mostró que el problema principal del laboratorio no es la falta de información, sino que esa información está repartida entre cuadernos, hojas de Excel y mensajes. Por eso el sistema se centra en un solo proceso y en tener un único lugar donde se registren las farmacias, los medicamentos y los pedidos con su estado.

Definir primero los actores y los requerimientos facilitó mucho el resto del diseño. Cada caso de uso, cada tabla y cada pantalla del prototipo se puede relacionar con un requerimiento concreto, y al revisar esa relación encontramos detalles que habrían sido difíciles de corregir después, como guardar el precio del medicamento en el detalle del pedido o desactivar farmacias en lugar de borrarlas para no perder su historial.

El modelo C4 nos ayudó a explicar la solución en distintos niveles: quién usa el sistema, cómo se divide en navegador, aplicación y base de datos, y cómo se organiza la aplicación por dentro en rutas, controladores, modelos y vistas. Esa organización es la misma que seguiremos al programar bajo el patrón MVC.

Con esta actividad queda un plano completo de la aplicación: el problema y sus objetivos, los requerimientos, la arquitectura, la base de datos con su script inicial, el prototipo de las pantallas y un plan de trabajo. Lo que sigue es implementar los módulos de farmacias y medicamentos en la Actividad 2, siempre que el docente apruebe el diseño y la tecnología propuesta.

Anexos

Anexo A. Archivos editables de los diagramas (carpeta diagramas/ del repositorio): casos-de-uso.drawio, c4-contexto.drawio, c4-contenedores.drawio, c4-componentes.drawio, er-conceptual.drawio, modelo-logico.drawio y modelo-fisico.drawio.

Anexo B. Script SQL inicial de la base de datos (database/script_inicial.sql).

Anexo C. Prototipo navegable en HTML (carpeta prototipos/, se abre desde index.html).